

Atividade 11 - Funções

Andrei Formiga

March 20, 2026

Sumário

1. Introdução	3
2. A linguagem Fun	4
2.1. Gramática	4
3. Análise léxica	6
4. Análise sintática	7
5. Análise semântica	8
5.1. Verificação de chamadas de função	8
5.2. Verificação de variáveis usadas em funções	8
6. Geração de código	10
6.1. Convenções de chamada no compilador Fun	11
6.1.1. Passagem de parâmetros na chamada	11
6.1.2. Implementação da função e variáveis locais	12
6.1.3. Acesso a variáveis locais	14
6.1.4. Resumo da implementação de funções	14
7. Funções no arquivo assembly	16
8. Artefato para entrega	17

1. Introdução

A linguagem Cmd da atividade anterior é Turing-completa, o que significa que ela pode ser usada para implementar qualquer algoritmo. Mas sabemos que em programas maiores e mais complicados, algumas operações se repetem e seria útil poder dividir o programa em sub-programas, na forma de funções.

Por exemplo, um programa mostrado para a linguagem Cmd foi um que calculava o valor absoluto do discriminante de uma equação de segundo grau. Em um programa maior, obter o valor absoluto é uma operação comum que pode ser repetida muitas vezes. Na linguagem Cmd esse cálculo teria que ser repetido toda vez que fosse necessário, aumentando o programa.

A próxima etapa do nosso projeto é adicionar funções à linguagem.

2. A linguagem Fun

A linguagem Fun adiciona funções à linguagem Cmd. Funções são declaradas com uma sintaxe similar a JavaScript, Go e Rust, usando a palavra chave `fun`, o nome da função, uma lista de parâmetros e o corpo da função. Por exemplo, uma função para calcular o valor absoluto poderia ser escrita da seguinte forma:

```
fun abs(x) {  
  var y = 0;  
  if x < 0 {  
    y = 0 - x;  
  } else {  
    y = x;  
  }  
  return y;  
}
```

O corpo da função pode ter zero ou mais declarações de variáveis locais (começando com a palavra-chave `var`), zero ou mais comandos, e deve ter uma expressão de resultado após a palavra-chave `return`.

Variáveis locais declaradas em funções (como `y` na função `abs` acima) são visíveis apenas dentro das funções onde são declaradas. Parâmetros são tratados como variáveis locais. Se uma variável local (ou parâmetro) tem o mesmo nome de uma variável global, a variável local tem precedência e esconde a variável global de mesmo nome. Por exemplo, dentro da função `abs` qualquer referência a `x` será uma referência ao parâmetro, mesmo que exista uma variável global `x` declarada antes da função `abs`.

Um programa na linguagem Fun é uma sequência de declarações (de variáveis globais e funções) seguida do bloco principal do programa. Para diferenciar as declarações de variáveis e de funções, as declarações de variáveis começam com a palavra-chave `var`. O bloco principal é marcado pela palavra-chave `main`.

2.1. Gramática

A gramática da linguagem Fun é mostrada abaixo:

```
<programa> ::= <decl>* 'main' '{' <cmd>* 'return' <exp> ';' '}'  
<decl> ::= <vardecl> | <fundecl>  
<fundecl> ::= 'fun' <ident> '(' <arglist>? ')'  
              '{' <vardecl>* <cmd>* 'return' <exp> ';' '}'  
<arglist> ::= <ident> | <ident> ',' <arglist>  
<vardecl> ::= 'var' <ident> '=' <exp> ';'   
<ident> ::= <letra><letra_digito>*  
<cmd> ::= <if> | <while> | <atrib>  
<if> ::= 'if' <exp> '{' <cmd>* '}' 'else' '{' <cmd>* '}'  
<while> ::= 'while' <exp> '{' <cmd>* '}'  
<atrib> ::= <ident> '=' <exp> ';'   
<exp> ::= <exp_a> (('{' | '>' | '=')* <exp_a>)*  
<exp_a> ::= <exp_m> (('+' | '-') <exp_m>)*  
<exp_m> ::= <prim> (('*' | '/') <prim>)*  
<prim> ::= <num> | <ident> | '(' <exp> ')' | <fun>  
<fun> ::= <ident> '(' <params>? ')'
```

```
<params> ::= <exp> | <exp> , <params>  
<num>    ::= <digito><digito>*
```

Esta gramática tem poucas alterações em relação à gramática da linguagem Cmd. Algumas mais importantes são:

- A produção `<programa>` inclui a palavra-chave `main` antes do bloco principal
- Declarações (`<decl>`) agora podem ser declarações de variáveis (`<vardecl>`) ou declarações de função (`<fundecl>`)
- Uma declaração de variáveis agora inclui a palavra-chave `var` no começo
- Declarações de função (`<fundecl>`) foram incluídas na gramática
- Nas expressões primárias (`<prim>`) foi adicionada uma nova possibilidade: uma chamada de função (`<fun>`)
- A regra para chamada de função (`<fun>`) foi adicionada

De resto, a maior parte da gramática continua a mesma em relação à gramática da linguagem Cmd.

Assim como nas atividades anteriores, a adição de funções à linguagem cria a necessidade de alterar todas as fases do compilador.

3. Análise léxica

A análise léxica da linguagem Fun muda muito pouco em relação à linguagem Cmd. Temos apenas um novo tipo de *token*, a vírgula, usada para separar os parâmetros formais na declaração de funções e os parâmetros reais na chamada de função.

Além desse novo tipo de *token*, três palavras-chave foram adicionadas à linguagem: `fun`, `var` e `main`.

O resto dos tipos de *token* e as palavras-chaves continuam iguais aos da linguagem Cmd.

4. Análise sintática

A gramática da linguagem Fun tem mais produções para reconhecer, mas nenhuma técnica nova é necessária.

Dois pontos que merecem alguma atenção são:

- O reconhecimento das listas de parâmetros (na declaração e chamada de funções)
- A diferenciação entre uma referência a uma variável e uma chamada de função

No primeiro ponto, a análise de uma lista de parâmetros formais deve ser feita em um *loop* que continua olhando o próximo *token* e reconhecendo um parâmetro até que o próximo *token* seja um fecha-parênteses. Em pseudo-código, esse processo seria algo similar ao seguinte:

```
analiseListaParametros:
  // o token anterior foi '('
  params = []
  tok = proximoToken()
  while tok ≠ EOF && tok ≠ ')':
    if tok.tipo = Identificador:
      params.adiciona(tok.lexema)
    tok = olharProximoToken()
    if tok = ',':
      proximoToken() // avanca o token da vírgula
      tok = proximoToken()

  if tok ≠ ')':
    error("Token ) esperado; fim da entrada encontrado")

  return params
```

Esta função reconhece uma vírgula extra no final da lista de parâmetros (por exemplo `fun f(x, y,)`), mas isso não é um grande problema. Uma função de reconhecimento um pouco mais sofisticada poderia detectar um problema nesse caso.

Para reconhecer uma lista de parâmetros reais na chamada de uma função, o processo é parecido, mas ao invés de guardar apenas o lexema do *token* identificador, é preciso chamar a função de reconhecer uma expressão (já que cada parâmetro real em uma chamada é uma expressão).

Com relação ao segundo ponto, diferenciar entre uma referência a uma variável e uma chamada de função é importante pois ambas as estruturas começam com um *token* identificador. Para diferenciar as duas, é necessário olhar o *token* seguinte após o identificador: se o próximo *token* for um abre-parêntese `(`, a expressão é uma chamada de função; caso contrário, é uma referência a uma variável.

5. Análise semântica

Até a linguagem Cmd a única tarefa de análise semântica era verificar se as referências a variáveis sempre usavam variáveis já declaradas. Isso é relativamente simples e precisa apenas de uma tabela de símbolos que registre quais variáveis já foram declaradas.

Na linguagem Fun temos algumas verificações novas relacionadas a funções, e uma complicação extra na verificação de variáveis.

5.1. Verificação de chamadas de função

Uma chamada de função deve verificar se a função foi declarada e se o número de parâmetros na chamada corresponde ao número de parâmetros formais na declaração. A verificação se a função já foi declarada é similar ao mesmo processo feito com variáveis. Se uma expressão contém uma chamada a uma função `f`, é preciso verificar se `f` existe na tabela de símbolos e se foi declarada como função (pois o nome poderia ser usado por uma variável).

As declarações em um programa devem ser processadas sequencialmente, e assim uma variável ou função pode referenciar variáveis e funções que foram declaradas anteriormente no texto do programa. Mas é preciso ter cuidado com o caso das funções recursivas. Por exemplo, a função a seguir calcula o n -ésimo número de Fibonacci usando uma função recursiva:

```
fun fib(n) {  
  var res = 0;  
  if n < 2 {  
    res = 1;  
  } else {  
    res = fib(n - 1) + fib(n - 2);  
  }  
  return res;  
}
```

No braço do `else`, o resultado é calculado chamando a função recursivamente. Neste ponto, se a função `fib` ainda não estiver na tabela de símbolos, o compilador vai rejeitar a chamada recursiva. O compilador deve permitir chamadas (diretamente) recursivas.

Se as declarações forem processadas na ordem que aparecem no programa, como sugerido, o compilador vai rejeitar funções que sejam mutuamente recursivas; isso não é um problema. Extender o compilador para aceitar funções mutuamente recursivas fica como exercício opcional.

A outra verificação em uma chamada de função é determinar se o número de parâmetros reais passados na chamada é igual ao número de parâmetros formais declarados pela função. Para isso, o número de parâmetros formais declarados deve estar disponível na tabela de símbolos; a análise apenas compara esse número com o número de parâmetros reais presentes na árvore da expressão.

5.2. Verificação de variáveis usadas em funções

Na verificação de variáveis usadas dentro do corpo de funções, é preciso levar em consideração a existência das variáveis locais e parâmetros da função.

Essas verificações requerem que mais informações sejam guardadas sobre cada símbolo (nome) usado pelo programa. Essas informações normalmente são guardadas na tabela de símbolos.

A tabela de símbolos precisa guardar qual tipo de entidade é referenciado por um símbolo: variável global, função ou variável local. A tabela também deve considerar o contexto, de forma que alguns símbolos só são considerados visíveis quando analisando código dentro de uma função.

Uma forma de fazer isso é ter tabelas de símbolos locais para cada função, e usar essas tabelas locais durante a análise do corpo da função. Por exemplo, se o programa contém uma função `f`, a análise do corpo da função `f` deve consultar a tabela local de `f` e, se não encontrar na tabela local, deve consultar a tabela global. As tabelas locais de cada função podem ser armazenadas na tabela global; para uma função `f`, sua entrada na tabela de símbolos global vai armazenar que `f` é um nome que denota uma função, junto com o número de parâmetros de `f` e sua tabela local.

Ao analisar o corpo da função `f`, o compilador entra no contexto da função `f`, usando a tabela de símbolos local de `f` e a tabela global durante a análise. Para verificar se uma variável já foi declarada, a análise consulta primeiro a tabela local; caso não encontre na tabela local, consulta a tabela global. Ao processar a declaração de uma nova variável local, a análise inclui a variável local na tabela local.

Para analisar se uma função já foi declarada, basta consultar diretamente a tabela global, já que não é possível declarar funções locais.

6. Geração de código

As mudanças na geração do código necessárias para compilar a linguagem Fun são obviamente relacionadas ao funcionamento das funções. Precisamos saber como fazer a chamada para a função e como implementar a função em si em *assembly*.

A chamada a uma função é um desvio incondicional, mas que de alguma forma *se lembra* do ponto de partida do desvio para retornar depois; ao final da execução da função chamada, um desvio deve ser feito para o endereço de retorno. Como chamadas de função podem ser aninhadas (a função `f` chama a função `g`, que chama a função `h`, etc), a forma natural de registrar o endereço de retorno é usando a pilha do sistema.

No conjunto de instruções x86-64 já existem duas instruções para chamada e retorno de funções: as instruções CALL e RET. A instrução CALL recebe um parâmetro; CALL empilha o endereço da instrução seguinte ao CALL na pilha do sistema, e faz um desvio incondicional para o endereço passado como parâmetro. RET não precisa de nenhum parâmetro; a instrução RET desempilha o endereço de retorno do topo da pilha e faz um desvio incondicional para o endereço desempilhado. Claro que para isso funcionar é preciso que o endereço de retorno esteja no topo da pilha quando o RET é executado.

Para fazer a chamada com CALL precisamos do endereço da função; para isso podemos usar um rótulo. Assim, o básico da definição e chamada de funções é simples: definimos funções usando um rótulo (vamos usar o mesmo nome dado pelo programador à função) e o código da função termina com uma instrução RET; quando a função é chamada em outra parte do código, o compilador emite uma instrução CALL para o rótulo correto.

Mas CALL e RET apenas definem a transferência de controle para o código de uma função. Uma função na linguagem Fun tem mais algumas características:

- Parâmetros podem ser passados do código chamador para a função chamada
- A função pode definir variáveis locais
- A função sempre retorna um valor para o chamador

Existem algumas formas diferentes de implementar essas funcionalidades. Por exemplo, para passar parâmetros para a função é possível usar registradores. Isso gera código eficiente, mas precisamos nos preocupar com o limite do número de registradores e o que fazer para que uma função não interfira nos valores dos registradores usados por outra função.

O criador de um compilador deve tomar as decisões de como implementar passagem de parâmetros, variáveis locais e passagem do valor de retorno e usar isso consistentemente no código gerado.

O conjunto de convenções usados na implementação de funções (definição e chamada) geralmente é chamado de *convenções de chamada* (*calling conventions* em inglês). Essas convenções podem variar entre linguagens, processadores e sistemas operacionais diferentes, ou até mesmo entre compiladores diferentes para a mesma linguagem. Uma boa alternativa na prática é seguir as convenções de chamada mais comuns no sistema operacional, por exemplo as convenções de chamada do System V em sistemas Unix; isso ajuda a interoperação entre a linguagem do compilador e outras linguagens já existentes no sistema.

Alguns conjuntos de convenções de chamada comuns em processadores de arquitetura x86 e x86-64 são descritos na seguinte entrada da Wikipedia: https://en.wikipedia.org/wiki/X86_calling_conventions^o.

Vamos adotar um conjunto de convenções de chamada que é relativamente simples de implementar e serve nossos propósitos.

6.1. Convenções de chamada no compilador Fun

Para implementar os parâmetros das funções e suas variáveis locais, usaremos novamente a pilha do sistema. Isso permite que cada chamada diferente da mesma função tenha suas próprias cópias dos parâmetros e variáveis locais, permitindo que funções sejam diretamente ou mutuamente recursivas.

(Versões antigas da linguagem FORTRAN alocavam endereços fixos de memória para os parâmetros e variáveis de uma sub-rotina; como consequência, não era possível escrever sub-rotinas ou funções recursivas.)

Para a passagem do valor de retorno da função, o mais simples é usar a mesma convenção usada em todas as expressões até agora: o resultado da função será retornado no registrador RAX.

6.1.1. Passagem de parâmetros na chamada

O código que chama uma função `f` deve, antes da instrução `CALL`, empilhar o valor de cada parâmetro da chamada. Para simplificar o acesso a esses parâmetros dentro da função (como veremos mais tarde), o recomendado é empilhar o valor dos parâmetros na ordem inversa, ou seja, o último parâmetro primeiro, depois o penúltimo, até o primeiro parâmetro que é empilhado por último. Em seguida, a instrução `CALL` vai empilhar o endereço de retorno na pilha.

Digamos que o compilador está gerando o código para uma chamada `f(11, 202)` e os parâmetros da função `f` são chamados de `x` e `y`, respectivamente. O código gerado será algo como:

```
mov $202, %rax    # exp. constante
push %rax         # empilha 2o parametro
mov $11, %rax     # exp. constante
push %rax         # empilha 1o parametro
call f
```

Logo após a execução da instrução `CALL`, a pilha terá o seguinte conteúdo:

```
+-----+ RSP
| ER    |
+-----+ 8
| x = 11 |
+-----+ 16
| y = 202 |
+-----+ 24
| ...   |
```

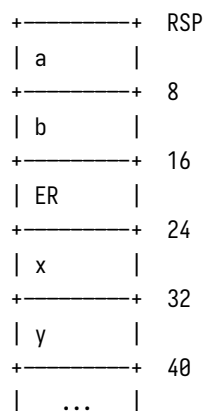
Nesse diagrama, ER é o endereço de retorno após a execução da função `f`. Como a pilha cresce com a redução do valor do registrador da pilha RSP, o parâmetro `x` estará na pilha 8 bytes depois do valor de RSP ($RSP+8$), e o parâmetro `y` estará em $RSP+16$. Esta é a situação na pilha logo que a execução da função `f` começa.

6.1.2. Implementação da função e variáveis locais

No início da função chamada, é preciso alocar espaço na pilha para as variáveis locais (se houver alguma). Digamos que `f` declare duas variáveis locais chamadas `a` e `b`. Cada variável ocupa 8 bytes, então precisamos alocar 16 bytes de espaço na pilha. Isso pode ser feito executando uma instrução PUSH para cada variável local, mas é mais rápido subtrair diretamente o espaço necessário:

```
f:
    sub $16, %rsp
```

Esse é o começo da função `f`, incluindo o rótulo e a instrução para alocar espaço para as variáveis locais. A instrução SUB cria os espaços para `a` e `b` na pilha. Agora a pilha vai ter a seguinte estrutura:



Esse espaço da pilha entre o topo atual (valor de RSP) e o endereço $RSP+40$ contém um bloco de informações da função `f`, o que é chamado de *registro de ativação* da função (em inglês normalmente chamado de *stack frame*, mas às vezes também chamado de *activation record*). No registro de ativação é possível acessar as variáveis locais e os parâmetros de `f`, tudo relativo ao valor de RSP. A variável local `a` está no endereço de RSP, `b` está no endereço $RSP+8$, o parâmetro `x` está em $RSP+24$, e por aí vai. Tudo isso pode ser calculado dependendo do número de variáveis locais e parâmetros da função.

Mas existe um problema: o código que nosso compilador produz para expressões usa a pilha; vamos lembrar que para uma expressão `M op N` o esquema de tradução é executar o código para a expressão `M`, empilhar o resultado em RAX, executar o código para `N`, desempilhar o valor salvo e em seguida realizar a operação `op`. Isso altera o valor de RSP e vai mudar os deslocamentos de todas as variáveis locais e todos os parâmetros, dificultando o acesso a essas variáveis.

Uma solução comum é usar o registrador RBP para salvar o endereço do começo do registro de ativação da função (chamado de *frame pointer*). Vamos seguir esse caminho, e portanto o

código de implementação de uma função deve começar fazendo duas coisas: alocar espaço na pilha, e em seguida copiar o valor de RSP para RBP:

```
f:
    sub $16, %rsp
    mov %rsp, %rbp
```

Mas isso cria a possibilidade de outro problema. Se a função `f` é chamada pelo bloco principal do programa, e não chama nenhuma outra função, nada errado acontece. Mas se `f` é chamada por uma função `g`, temos que considerar que `g` também está usando RBP como referência para seu registro de ativação. Ao copiar o valor de RSP em RBP, o valor anterior de RBP se perde; da mesma forma, se `f` chama outra função (ou ela mesma), o valor de RBP usado por `f` será perdido.

Para resolver esse problema, vamos usar a pilha mais uma vez. Vamos salvar o valor anterior de RBP antes de alocar espaço para as variáveis locais e copiar o valor de RSP:

```
f:
    push %rbp
    sub $16, %rsp
    mov %rsp, %rbp
```

A estrutura da pilha agora fica um pouco diferente, mas ainda é fácil de calcular os deslocamentos de todas as variáveis locais:

+-----+	RSP
a	
+-----+	8
b	
+-----+	16
RBP	
+-----+	24
ER	
+-----+	32
x	
+-----+	40
y	
+-----+	48
...	

Ao final da função `f` é preciso desfazer o que foi feito no início, para deixar o endereço de retorno (ER) no topo da pilha para a instrução `RET`:

```
...
    add $16, %rsp
    pop %rbp
    ret
```

Após a instrução `RET`, o programa volta para executar a instrução seguinte após o `CALL`, mas a pilha ainda vai conter os parâmetros `x` e `y`. É preciso removê-los da pilha, o que pode ser feito com várias instruções `POP` ou apenas adicionando 16 ao valor de RSP.

6.1.3. Acesso a variáveis locais

Ao gerar código para uma função, expressões que sejam referências a variáveis precisam agora diferenciar se a variável é global ou local. Se for global, continua como antes, usando um símbolo criado na seção BSS (com mesmo nome da variável global).

Se for uma variável local, é preciso fazer o acesso (seja leitura ou escrita) ao local no registro de ativação da função, usando o valor de RBP como base.

Ao gerar o código da função, é possível calcular qual o deslocamento de cada variável local e cada parâmetro no registro de ativação da função. No exemplo da função `f` acima, sabemos que `b` está em `RBP+8` e `y` está em `RBP+40`.

Para acessar esse local de memória utilizamos o modo indireto por registro; ou seja, não queremos ler ou escrever o valor de RBP, mas o valor apontado por RBP na memória (ou RBP com deslocamento). Para armazenar o valor 15 na variável local `b` do exemplo acima, usamos a instrução

```
mov $15, 8(%rbp)
```

O operando `8(%rbp)` significa *o valor na memória com endereço RBP+8*. A instrução MOV acima diz: armazene o valor 15 na memória no endereço RBP+8. Já se quisermos copiar o valor do parâmetro `y` para RAX, a instrução é

```
mov 40(%rbp), %rax
```

Funciona de modo similar: pegue o valor na memória com endereço RBP+40, e copie para RAX.

Normalmente, a tabela de símbolos é usada para guardar informações sobre as variáveis, o que inclui o tipo da variável (local ou global) e o deslocamento da variável no registro de ativação da função, caso a variável seja local.

6.1.4. Resumo da implementação de funções

Resumindo, na chamada de uma função, deve ser executada a seguinte sequência:

1. Calcular o valor de cada parâmetro e empilha-los, na ordem inversa do último parâmetro até o primeiro
2. Fazer a chamada em si com a instrução CALL usando o rótulo da função
3. Remover os parâmetros da pilha após a chamada, para retornar a pilha ao seu estado anterior

Para gerar o corpo de uma função com `L` variáveis locais:

1. Gerar o rótulo da função
2. Empilhar o valor anterior de RBP
3. Subtrair `L` vezes 8 a RSP para alocar espaço para variáveis locais
4. Copiar valor de RSP para RBP
5. Gerar o código que calcula e armazena o valor de cada variável local
6. Gerar o código para os comandos
7. Gerar o código para a expressão de resultado (que deixa o resultado em RAX)

8. Somar L vezes 8 a RSP para remover o espaço das variáveis locais
9. Desempilhar o valor anterior de RBP
10. Retornar com RET

Por exemplo, seja a função `f` abaixo:

```
fun f(x) {  
  var y = x * x + 7;  
  var z = y * x - 9;  
  return y * z;  
}
```

Digamos que `f` seja chamada com um parâmetro constante de valor 42. A sequência de instruções para chamada de `f` é:

```
mov $42, %rax  
push %rax      # parametro na pilha  
call f  
add $8, %rsp
```

E o compilador vai gerar o seguinte código (resumido) para a função `f`:

```
f:  
  push %rbp  
  sub $16, %rsp  
  mov %rsp, %rbp  
  # codigo que calcula y (omitido)  
  mov %rax, 0(%rbp)  
  # codigo que calcula z (omitido)  
  mov %rax, 8(%rbp)  
  # expressao de resultado  
  mov 0(%rbp), %rax  
  push %rax  
  mov 8(%rbp), %rax  
  pop %rbx  
  mul %rbx      # resultado em RAX  
  add $16, %rbp  
  pop %rbp  
  ret
```

Dessa forma, é possível implementar funções com número arbitrário de parâmetros e variáveis locais. Essas funções podem ser diretamente ou mutuamente recursivas.

7. Funções no arquivo assembly

Para cada função do programa de entrada, o compilador deve gerar um rótulo com o mesmo nome da função, o código de prólogo e o código de epílogo, além do código interno da função. O código dessas funções, incluindo seus rótulos, deve ser colocado no arquivo assembly na seção TEXT, antes ou depois do código do programa principal (marcado com o rótulo `_start`).

O modelo do arquivo *assembly* agora tem três partes que serão preenchidas pelo compilador: a parte para declaração das variáveis globais, a parte do programa principal, e a parte das funções:

```
# Modelo para arquivo de saída

.section .bss
## diretivas para as variaveis

.section .text
.globl _start

_start:
## codigo gerado para o programa principal

call imprime_num
call sair

## codigo gerado para as funcoes

.include "runtime.s"
```

8. Artefato para entrega

Cada grupo deve entregar o compilador para a linguagem Fun ou alguma variante da linguagem. No caso da linguagem implementada pelo compilador ser diferente, de alguma forma, da linguagem Cmd como especificada neste guia, a documentação do projeto deve deixar claras as diferenças em relação ao que está no guia.

O projeto deve incluir programas de teste que contenham funções em algumas situações de interesse, por exemplo funções recursivas, funções que chamam outras funções, funções com e sem variáveis locais, com ou sem parâmetros, etc. A documentação deve descrever claramente como executar os testes incluídos.